

Smart CNC Controller

& Path Generator

Complete Project Documentation

Version 4.0 - June 2026

*An intelligent 2-axis CNC path generation system combining
Natural Language Processing, Mathematical Expression Parsing,
and Computer Vision for precise toolpath creation.*

Technologies: Python | Ollama | Tkinter | OpenCV | Telegram Bot API

Hardware: AVR/Arduino CNC Machines

Table of Contents

Chapter 1	Project Overview & Architecture	3
Chapter 2	How Ollama Works - NLP Engine	5
Chapter 3	How Tkinter Works - GUI Simulator	8
Chapter 4	Telegram Bot Integration	11
Chapter 5	Mathematical Expression Parser	14
Chapter 6	Computer Vision & Image Tracing	17
Chapter 7	AVR/Arduino Hardware Integration	19
Chapter 8	Code Deep Dive - File by File	21
Chapter 9	Docker Deployment Guide	28
Chapter 10	API Reference & Commands	30
Appendix A	Complete Preset Shapes Library	32
Appendix B	Troubleshooting Guide	34

Chapter 1: Project Overview & Architecture

1.1 Introduction

The Smart CNC Controller is an intelligent 2-axis CNC path generation system that bridges the gap between human language and machine precision. By combining Natural Language Processing (NLP) via Ollama, robust mathematical expression parsing, and computer vision, this system allows users to create precise toolpaths using natural language commands, mathematical equations, or even by tracing images.

The system is designed as a Telegram bot interface, making it accessible from any device with Telegram installed. Users can simply type commands like "Draw a heart shape" or "Plot $y = \sin(x)$ " and the system will automatically generate the corresponding CNC paths, visualize them in a Tkinter simulator, and export them as C header files for AVR/Arduino hardware.

1.2 Core Features

- Intelligent NLP Parser using Ollama qwen2.5:0.5b model
- Advanced regex-based mathematical expression parser
- 23+ preset mathematical curves and shapes
- Image-to-CNC path tracing using OpenCV
- Real-time visualization with Tkinter-based simulator
- AVR-ready C header file export (Custom.h)
- Telegram bot interface with user authentication
- Docker deployment support

1.3 System Architecture

The system follows a modular architecture with clear separation of concerns:

1. Interface Layer (Telegram Bot) - Handles user interaction and authentication
2. Processing Layer (NLP + Math Parser) - Converts user input to parametric equations
3. Computation Layer (NumPy) - Generates coordinate arrays
4. Visualization Layer (Tkinter) - Renders paths in real-time
5. Export Layer (C Header Generator) - Creates hardware-ready files

1.4 Project File Structure

creating_CNC/	
main_auto.py	- Entry point, Telegram bot logic, NLP pipeline
new.py	- Core engine: Draw class, curve generation, image tracing
CNC_simulation.py	- Tkinter-based CNC path visualizer
create_c_array.py	- Exports numpy arrays to C header files
Custom.h	- Generated C header with path data
requirements.txt	- Python dependencies
chat_history.json	- Conversation memory for context-aware responses
Dockerfile	- Docker deployment configuration
README.md	- Project documentation

Chapter 2: How Ollama Works - NLP Engine

2.1 What is Ollama?

Ollama is a lightweight framework for running large language models (LLMs) locally on your machine. It provides a simple API for interacting with various open-source models, including Llama, Mistral, and Qwen. In this project, we use Ollama to run the qwen2.5:0.5b model, which is a compact but capable language model optimized for instruction following.

2.2 Installation & Setup

```
# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh

# Pull the qwen2.5:0.5b model
ollama pull qwen2.5:0.5b

# Start Ollama server
ollama serve

# Test the model
ollama run qwen2.5:0.5b "Hello, how are you?"
```

2.3 How Ollama Integration Works

The project integrates Ollama through the official Python library. The integration works through two main functions:

1. `_ollama_parse(text)` - Converts natural language to parametric equations
2. `ollama_respond(text)` - Handles general conversation and guidance

NLP Parsing Pipeline

When a user sends a message, the system follows this pipeline:

1. First, check if the text matches a preset shape name
2. If not, try to parse it as a mathematical equation using regex
3. If that fails, send the text to Ollama with a specialized system prompt
4. Ollama returns JSON with parametric equations
5. If Ollama returns "NOT_DRAW", treat it as a conversation message

2.4 System Prompt Engineering

The system prompt for Ollama is carefully crafted to ensure reliable output. It instructs the model to:

- Return ONLY valid JSON (no markdown, no explanation)
- Use Python syntax for mathematical expressions
- Include specific fields: `x_expr`, `y_expr`, `t_start`, `t_end`, `n_points`, `r`
- Return "NOT_DRAW" for non-drawing requests

```
_SYSTEM_DRAW_PROMPT = """
You are a CNC math parser. Convert user input to parametric equations.
```

RULES:

1. Return ONLY valid JSON:

```
{ "x_expr": "...", "y_expr": "...", "t_start": 0, "t_end": 6.28,
  "n_points": 300, "r": 10 }
```

2. Python syntax: t^2 , $\sin(t)$, $\cos(t)$, $\sqrt{t}$, $\exp(t)$, π

3. If NOT a draw request, reply: NOT_DRAW

```
"""
```

2.5 Response Parsing

The Ollama response is parsed with error handling:

1. Check if response contains "NOT_DRAW"
2. Remove markdown code block markers if present
3. Parse JSON and validate required fields
4. Return None on any error (falls back to conversation mode)

```
def _ollama_parse(text):
    try:
        resp = ollama.chat(
            model=OLLAMA_MODEL,
            messages=[
                {"role": "system", "content": _SYSTEM_DRAW_PROMPT},
                {"role": "user", "content": text},
            ],
        )
        raw = resp["message"]["content"]
        if "NOT_DRAW" in raw:
            return None
        cleaned = re.sub(r"^```(?:json)?\s*\n?", "", raw.strip())
        cleaned = re.sub(r"\s*```$", "", cleaned)
        data = json.loads(cleaned)
        return data
    except Exception:
        return None
```

2.6 Conversation Memory

The system maintains conversation history in `chat_history.json`. This allows Ollama to provide context-aware responses. The history is limited to the last 10 messages (5 pairs of user-assistant exchanges) to prevent context overflow.

2.7 Model Selection Rationale

The qwen2.5:0.5b model was chosen for several reasons:

1. Small size (0.5 billion parameters) - runs on consumer hardware
2. Fast inference - suitable for real-time interaction
3. Good at following instructions - reliable JSON output
4. Supports mathematical reasoning - understands equations
5. Free and open source - no API costs

Chapter 3: How Tkinter Works - GUI Simulator

3.1 Introduction to Tkinter

Tkinter is Python's standard GUI (Graphical User Interface) library. It provides a powerful set of tools for creating windows, dialogs, buttons, and drawing canvases. In this project, Tkinter is used to create a real-time CNC path visualizer that shows exactly what the machine will do before sending it to hardware.

3.2 Tkinter Architecture

Tkinter follows a hierarchical widget-based architecture:

1. Root Window (Tk) - The main application window
2. Canvas Widget - Drawing area for paths and shapes
3. Event Loop (mainloop) - Handles user interactions and updates

3.3 The CNC Simulation Class

The CNC_simulation.py file defines a comprehensive CNC simulation class that wraps Tkinter functionality into a user-friendly API. Key features include:

- Coordinate system transformation (real coords to canvas pixels)
- Animated path drawing with configurable speed
- Grid and axis visualization
- Tool position tracking (red dot marker)
- Multiple drawing primitives: point, segment, line_to, move_to
- PostScript export capability

3.4 Coordinate System Transformation

The simulator converts real-world CNC coordinates to canvas pixel coordinates using a transformation function. The canvas has its origin at the top-left corner, while CNC coordinates have the origin at the center with Y-axis pointing up.

```
def _to_canvas_coords(self, x, y):
    canvas_x = (x - self.x_min) * self.x_scale
    canvas_y = self.height - (y - self.y_min) * self.y_scale
    return canvas_x, canvas_y
```

3.5 Animation System

The animation system provides smooth, real-time visualization of path drawing. It uses frame-based interpolation to animate tool movement:

1. Calculate number of steps based on delay or speed
2. Interpolate between start and end points
3. Update tool position marker at each step
4. Draw line segments progressively
5. Sleep between frames for smooth animation

```
def _animate_segment(self, x1, y1, x2, y2, color='blue', width=2):
    # Calculate number of steps (target 60 FPS)
```

```
num_steps = max(int(self.draw_delay * 60), 10)

for i in range(num_steps + 1):
    t = i / num_steps
    curr_x = x1 + t * (x2 - x1)
    curr_y = y1 + t * (y2 - y1)

    self.current_x = curr_x
    self.current_y = curr_y
    self._update_tool_position()

    # Draw line segment
    if i > 0:
        self.canvas.create_line(prev_cx, prev_cy, curr_cx, curr_cy,
                                fill=color, width=width)

    self.canvas.update()
    time.sleep(self.draw_delay / num_steps)
```

3.6 Grid and Axes Drawing

The simulator draws a grid system with configurable spacing. Vertical and horizontal grid lines are drawn with dashed styling. The X and Y axes are drawn as solid black lines, with an origin point marker.

3.7 Thread Safety

Since Tkinter is not thread-safe, the project uses a queue-based architecture. The Telegram bot runs in a separate thread and places draw requests into a queue. The main thread (which owns Tkinter) drains the queue and executes drawing operations. This ensures all GUI operations happen in the main thread.

```
# Main thread owns Tkinter - drain draw queue forever
while True:
    try:
        data = _draw_queue.get(timeout=1)
        # Execute drawing in main thread
        new.solution(xc, yc, t, title)
    except queue.Empty:
        pass
```

Chapter 4: Telegram Bot Integration

4.1 Telegram Bot API

Telegram provides a powerful Bot API that allows developers to create automated chatbots. The pyTelegramBotAPI library (telebot) provides a Python wrapper for this API, making it easy to handle messages, commands, and media.

4.2 Bot Configuration

The bot is configured with three key parameters:

1. API Token - Obtained from BotFather on Telegram
2. ALLOWED_USERS - List of authorized Telegram user IDs
3. CONNECT_TIMEOUT - Network timeout (30 seconds)

```
API = "8690254124:AAG4hFS89yHbsEcNT3Wsfoa6io1jLVUAGgI"
bot = telebot.TeleBot(token=API)
apihelper.CONNECT_TIMEOUT = 30
ALLOWED_USERS = [8058658801, 1667679794]
```

4.3 Security & Authentication

The system implements a simple but effective authentication mechanism. Each incoming message is checked against the ALLOWED_USERS list. Unauthorized users receive a message with their Telegram ID, which they can share with the admin to gain access.

```
def is_authorized(message):
    uid = message.from_user.id
    if uid not in ALLOWED_USERS:
        bot.reply_to(message, f"Access denied. Send your ID to admin: {uid}")
        return False
    return True
```

4.4 Message Handlers

The bot uses decorator-based message handlers to process different types of input:

1. /start, /help - Display welcome message and available commands
2. /myid - Return user's Telegram ID
3. /demo - Run sequence of 5 preset shapes
4. /draw x|y|pts - Manual parametric drawing mode
5. /<shape_name> - Draw specific preset shape
6. Photo messages - Convert image to CNC paths
7. Text messages - Parse with NLP pipeline

4.5 Command Registration

Bot commands are registered both via decorators and via the set_my_commands API. This provides autocomplete suggestions in the Telegram interface.

```
bot.set_my_commands([
    telebot.types.BotCommand("myid", "Get your Telegram ID"),
```



```
telebot.types.BotCommand("start", "Show help & commands"),
telebot.types.BotCommand("demo", "Run 5 preset shapes"),
telebot.types.BotCommand("heart", "Heart curve"),
telebot.types.BotCommand("circle", "Circle"),
# ... more commands

])
```

4.6 Photo Handling

When a user sends a photo, the bot:

1. Downloads the photo using `bot.get_file()` and `bot.download_file()`
2. Saves it as `input_image.jpg`
3. Creates a Draw instance and calls `draw_cartoon()`
4. OpenCV processes the image and generates CNC paths
5. The paths are visualized in the Tkinter simulator

4.7 Polling vs Webhooks

This project uses long polling instead of webhooks. Polling is simpler to set up and doesn't require a public server or HTTPS certificate. The bot runs in a daemon thread with a 60-second timeout.

```
threading.Thread(
    target=lambda: bot.polling(
        none_stop=True,
        timeout=60,
        long_polling_timeout=60
    ),
    daemon=True
).start()
```

Chapter 5: Mathematical Expression Parser

5.1 Overview

The mathematical expression parser is a robust regex-based system that converts human-friendly math expressions into NumPy-evaluable strings. It handles implicit multiplication, trigonometric functions, and various notation styles.

5.2 Supported Syntax

The parser supports various input formats:

- Standard notation: $y = \sin(x)$, $y = x^2 + 5$
- Implicit multiplication: $2x$ becomes $2*x$
- Function calls: $\sin x$ becomes $\sin(x)$, $\sqrt{x}$ works
- Parentheses groups: $(x+1)(x-1)$ becomes $(x+1)*(x-1)$
- Power notation: x^2 becomes $x**2$
- Both x and y as dependent variables

5.3 Parsing Pipeline

The regex parser follows these steps:

1. Match equation pattern: $y = \dots$ or $x = \dots$
2. Infer dependent variable if no = sign
3. Replace $\wedge$ with $**$ (power operator)
4. Add parentheses to trig functions: $\sin x \rightarrow \sin(x)$
5. Add implicit multiplication: $2x \rightarrow 2*x$
6. Replace dependent variable with t (parameter)
7. Replace unknown constants with 1
8. Auto-detect range based on function type
9. Validate by test evaluation

5.4 Range Detection

The parser automatically determines the parameter range based on the functions present:

1. Trigonometric functions ($\sin$, $\cos$, $\tan$): -2π to 2π
2. Logarithmic/Square root: 0.1 to 10
3. Polynomial/Linear: -4 to 4

5.5 Safe Evaluation

All mathematical expressions are evaluated in a restricted environment to prevent code injection. The `eval()` function is called with `__builtins__: {}` and a whitelist of safe functions and constants.

```
_SAFE_MATH = {
    "sin": np.sin, "cos": np.cos, "tan": np.tan,
    "sqrt": np.sqrt, "exp": np.exp, "log": np.log,
    "abs": np.abs, "pi": np.pi, "e": np.e,
}
```

```
def _eval_expr(expr, t, r=10.0):  
    safe = {"t": t, "r": r, "np": np, **_SAFE_MATH}  
    val = eval(expr, {"__builtins__": {}}, safe)  
    return val
```

5.6 Preset Lookup System

Before attempting regex parsing, the system checks if the input matches any preset shape names or NLP keywords. This provides instant response for common shapes without needing the LLM.

Chapter 6: Computer Vision & Image Tracing

6.1 OpenCV Integration

OpenCV (Open Source Computer Vision Library) is used to convert user-provided images into CNC toolpaths. The `image_to_cnc_coords()` function implements a pipeline that extracts contours and simplifies them for CNC machining.

6.2 Image Processing Pipeline

The image tracing follows these steps:

1. Load image in grayscale mode
2. Apply Gaussian Blur to remove noise
3. Apply binary threshold (127) to create B&W image
4. Find external contours using `cv2.findContours()`
5. Simplify each contour using Douglas-Peucker algorithm
6. Scale coordinates to CNC range (-scale to +scale)
7. Return list of simplified paths

6.3 Contour Simplification

The Douglas-Peucker algorithm (`cv2.approxPolyDP`) is key to the "cartoon" effect. The epsilon parameter controls simplification:

- Higher epsilon = more simplified/blocky
- Lower epsilon = more detailed/smooth

In this project, $\text{epsilon} = 0.02 * \text{arcLength}$ provides good balance.

```
# THE CARTOON TRICK
epsilon = 0.02 * cv2.arcLength(cnt, True)
approx = cv2.approxPolyDP(cnt, epsilon, True)

# Only keep shapes with more than 2 points
if len(approx) > 2:
    points = approx.reshape(-1, 2).astype(float)
    # Scale to CNC range
    points[:, 0] = (points[:, 0] / img.shape[1] - 0.5) * (scale * 2)
    points[:, 1] = (0.5 - points[:, 1] / img.shape[0]) * (scale * 2)
```

6.4 Coordinate Scaling

Image pixel coordinates are converted to CNC coordinates with origin at center. The X-axis is centered horizontally and the Y-axis is inverted (image Y increases downward, CNC Y increases upward).

Chapter 7: AVR/Arduino Hardware Integration

7.1 C Header File Export

The `create_c_array.py` module converts Python/NumPy arrays into C header files suitable for AVR/Arduino projects. The export includes:

- `NUM_POINTS` constant (number of path points)
- `path_x[]` float array (X coordinates)
- `path_y[]` float array (Y coordinates)
- `path_z[]` float array (Z coordinates)

7.2 Export Format

```
#define NUM_POINTS 300

float path_x[] = {
    0.500000f,
    0.503775f,
    0.506681f,
    // ... more points
};

float path_y[] = {
    0.000000f,
    0.021185f,
    // ... more points
};
```

7.3 Arduino Integration Example

To use the generated header in an Arduino project:

```
#include "Custom.h"

void executePath() {
    for(int i = 0; i < NUM_POINTS; i++) {
        moveTo(path_x[i], path_y[i]);
        // Optionally control Z-axis:
        // setZ(path_z[i]);
    }
}

void setup() {
    // Initialize stepper motors
    // Set microstepping, speed, etc.
}

void loop() {
    executePath();
}
```

```
    delay(5000); // Wait 5 seconds  
}
```

7.4 float Precision

Coordinates are exported as float with 6 decimal places (e.g., 0.500000f). This provides sufficient precision for most CNC applications while keeping file sizes reasonable.

Chapter 8: Code Deep Dive - File by File

8.1 main_auto.py - Entry Point

This is the main entry point that orchestrates all components. Key sections:

1. Configuration (lines 1-16) - API tokens, model selection, user whitelist
2. Preset Shapes (lines 18-47) - Dictionary of 23+ mathematical curves
3. NLP Keywords (lines 49-71) - Keyword-to-preset mapping
4. Safe Math Eval (lines 73-89) - Restricted evaluation environment
5. Render Queue (lines 91-95) - Thread-safe drawing interface
6. NLP Pipeline (lines 97-223) - Ollama + regex parsing
7. Security (lines 272-279) - User authentication
8. Handlers (lines 281-461) - Telegram message processing
9. Main Loop (lines 442-462) - Tkinter queue processor

8.2 new.py - Core Engine

Contains the Draw class and curve generation logic:

1. image_to_cnc_coords() - OpenCV image processing
2. solution() - Renders curves and exports to C header
3. Draw class - 20+ curve generation methods

Each curve method follows the pattern:

- Generate parameter t using np.linspace
- Calculate x and y coordinates using NumPy operations
- Call solution() to render and export

8.3 CNC_simulation.py - Visualizer

Complete Tkinter-based visualization system:

1. CNC class - Main simulation engine
2. Coordinate transformation - Real to canvas coords
3. Animation system - Frame-based interpolation
4. Drawing primitives - point, segment, line_to, move_to
5. Grid/axes rendering - Visual reference
6. Helper function - plot_path() for multi-point paths

8.4 create_c_array.py - Exporter

Simple but effective C header generator:

1. Takes x, y, z numpy arrays as input
2. Writes #define NUM_POINTS
3. Writes path_x[], path_y[], path_z[] float arrays
4. Uses 6 decimal place precision

Chapter 9: Docker Deployment Guide

9.1 Dockerfile Overview

The Dockerfile creates a self-contained environment with all dependencies. Key components:

1. Base image: Python 3.11-slim
2. System packages: tk-dev, OpenCV dependencies
3. Ollama installation and model download
4. Python dependencies from requirements.txt
5. Startup script that runs both Ollama and the bot

9.2 Building the Image

```
# Build the Docker image
docker build -t cnc-controller .

# This will:
# 1. Pull Python 3.11-slim base image
# 2. Install system dependencies
# 3. Install Python packages
# 4. Install Ollama
# 5. Download qwen2.5:0.5b model
# 6. Copy application files
```

9.3 Running the Container

```
# Run with Telegram bot token
docker run -d --name cnc-bot \
    -e TELEGRAM_TOKEN="your_token_here" \
    cnc-controller

# Run with GPU support (for faster Ollama)
docker run -d --gpus all --name cnc-bot cnc-controller

# View logs
docker logs -f cnc-bot
```

9.4 Docker Compose

For production deployment, consider using docker-compose:

```
version: "3.8"
services:
  cnc-bot:
    build: .
    container_name: cnc-controller
    restart: unless-stopped
    environment:
      - TELEGRAM_TOKEN=${TELEGRAM_TOKEN}
```



```
volumes:
  - ./data:/app/data
ports:
  - "11434:11434"
```

9.5 Environment Variables

Important environment variables:

- TELEGRAM_TOKEN: Bot API token
- ALLOWED_USERS: Comma-separated user IDs
- OLLAMA_MODEL: Model to use (default: qwen2.5:0.5b)
- DISPLAY: For headless systems (use Xvfb)

Chapter 10: API Reference & Commands

10.1 Telegram Bot Commands

Available commands:

/start	Show welcome message and available commands
/help	Display help information
/myid	Return your Telegram user ID
/demo	Run 5 preset shapes in sequence
/draw x y pts t0 t1	Draw custom parametric equation
/circle [radius]	Draw circle (default r=10)
/heart	Draw heart curve
/butterfly	Draw butterfly curve
/spiral	Draw Archimedean spiral
/rose	Draw 5-petal rose
/cardioid	Draw cardioid
/lissajous	Draw Lissajous figure-8
/astroid	Draw astroid
/star	Draw 5-point star
/infinity	Draw infinity symbol
/parabola	Draw parabola $y=x^2$
/sine	Draw sine wave
/lemniscate	Draw lemniscate of Bernoulli
/cycloid	Draw cycloid
/deltoid	Draw deltoid

10.2 Natural Language Examples

You can also type naturally:

- "Draw a heart shape for my girlfriend"
- "Make a circle with radius 15"
- "Plot a butterfly curve"
- " $y = \sin(x)$ " or " $y = x^2 + 3x$ "
- " $x = \cos(y)$ "

10.3 Image Mode

Send any photo to the bot and it will:

1. Trace the image contours
2. Simplify to "cartoon" style
3. Convert to CNC toolpaths
4. Visualize in the simulator

10.4 Draw Class Methods

The Draw class in new.py provides these curve methods:

```
Draw.x, y, r.circle(radius)
Draw.x, y, r.heart_curve()
Draw.x, y, r.petal_rose(radius)
Draw.x, y, r.lissajous(radius)
Draw.x, y, r.butterfly(radius)
Draw.x, y, r.spiral(radius)
Draw.x, y, r.cardioid(radius)
Draw.x, y, r.astroid(radius)
Draw.x, y, r.epitrochoid(radius)
Draw.x, y, r.hypotrochoid(radius)
Draw.x, y, r.rhodonea(radius, petals)
Draw.x, y, r.limacon(radius)
Draw.x, y, r.cycloid(radius)
Draw.x, y, r.deltoid(radius)
Draw.x, y, r.logarithmic_spiral(radius)
Draw.x, y, r.lemniscate(radius)
Draw.x, y, r.sine_wave(amplitude, frequency, length)
Draw.x, y, r.witch_of_agnesi(radius)
Draw.x, y, r.draw_image(image_path)
```

Chapter A: Complete Preset Shapes Library

All 23+ preset shapes with their parametric equations:

circle	$x = r \cdot \cos(t), y = r \cdot \sin(t)$	<i>[0 to 2pi]</i>
heart	$x = 16 \cdot \sin^3(t), y = 13 \cdot \cos(t) - 5 \cdot \cos(2t) - 2 \cdot \cos(3t) - \cos(4t)$	<i>[0 to 2pi]</i>
rose	$x = r \cdot \cos(5t) \cdot \cos(t), y = r \cdot \cos(5t) \cdot \sin(t)$	<i>[0 to pi]</i>
lissajous	$x = r \cdot \sin(t), y = r \cdot \sin(2t)$	<i>[0 to 2pi]</i>
butterfly	$x = r \cdot \sin(t) \cdot \exp, y = r \cdot \cos(t) \cdot \exp$	<i>[0 to 10pi]</i>
spiral	$x = (r/10) \cdot t \cdot \cos(t), y = (r/10) \cdot t \cdot \sin(t)$	<i>[0 to 8pi]</i>
cardioid	$x = r \cdot (1 + \cos(t)) \cdot \cos(t), y = r \cdot (1 + \cos(t)) \cdot \sin(t)$	<i>[0 to 2pi]</i>
astroid	$x = r \cdot \cos^3(t), y = r \cdot \sin^3(t)$	<i>[0 to 2pi]</i>
epitrochoid	$x = (R+rs) \cdot \cos(t) - d \cdot \cos((R+rs)/rs \cdot t)$	<i>[0 to 2pi]</i>
hypotrochoid	$x = (R-rs) \cdot \cos(t) + d \cdot \cos((R-rs)/rs \cdot t)$	<i>[0 to 2pi]</i>
rhodonea	$x = r \cdot \cos(kt) \cdot \cos(t), y = r \cdot \cos(kt) \cdot \sin(t)$	<i>[0 to pi]</i>
limacon	$x = r \cdot (1 + 0.5 \cdot \cos(t)) \cdot \cos(t)$	<i>[0 to 2pi]</i>
cycloid	$x = r \cdot (t - \sin(t)), y = r \cdot (1 - \cos(t))$	<i>[0 to 4pi]</i>
deltoid	$x = r \cdot (2 \cdot \cos(t) + \cos(2t))$	<i>[0 to 2pi]</i>
lemniscate	$x = r \cdot \cos(t) / (1 + \sin^2(t))$	<i>[0 to 2pi]</i>
sine	$x = t, y = r \cdot \sin(3t)$	<i>[-10 to 10]</i>
parabola	$x = t, y = t^2$	<i>[-4 to 4]</i>
logspiral	$x = (r/20) \cdot e^{(0.2t)} \cdot \cos(t)$	<i>[0 to 4pi]</i>
infinity	$x = r \cdot \sin(t), y = r \cdot \sin(t) \cdot \cos(t) / (1 + \sin^2(t))$	<i>[0 to 2pi]</i>
star	$x = r \cdot \cos(t) \cdot (1 + 0.3 \cdot \cos(5t))$	<i>[0 to 2pi]</i>
rainbow	$x = (r/3) \cdot t \cdot \cos(t), y = (r/3) \cdot t \cdot \sin(t)$	<i>[0 to 12pi]</i>

Chapter B: Troubleshooting Guide

Common Issues & Solutions

Issue: Ollama not starting

Solution: Ensure ollama serve is running. Check with: `curl http://localhost:11434/api/tags`

Issue: Model not found

Solution: Pull the model: `ollama pull qwen2.5:0.5b`

Issue: Telegram bot not responding

Solution: Check API token and network connection. Verify ALLOWED_USERS list.

Issue: Tkinter error in Docker

Solution: Install xvfb for headless display: `apt-get install xvfb`

Issue: OpenCV import error

Solution: Install system dependencies: `apt-get install libgl1-mesa-glx`

Issue: Permission denied on Custom.h

Solution: Check file permissions: `chmod 644 Custom.h`

Issue: Memory issues with large images

Solution: Resize images before sending to bot

Issue: Animation too fast/slow

Solution: Adjust `draw_delay` parameter in CNC class

Performance Tips

- Use fewer points (100-200) for faster simulation
- Increase `draw_delay` for slower, more visible animation
- Run Ollama on GPU for faster NLP responses
- Use Docker with `--gpus all` for GPU acceleration
- Pre-download models in Docker build for faster startup